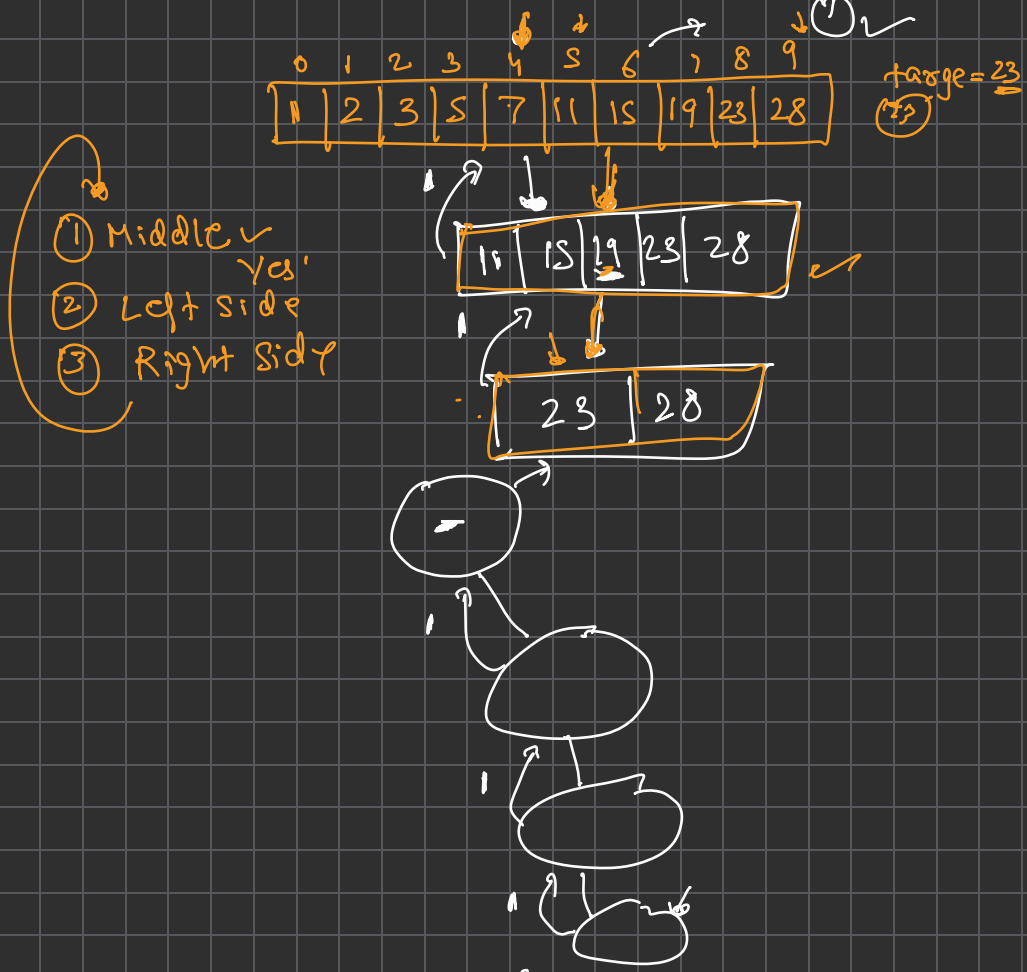
Binary Search ⇒ Recursion

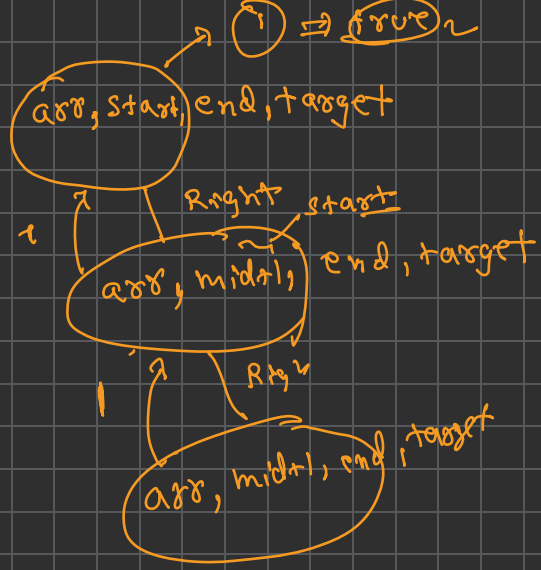
target = 12

0	1	2	3	4	5
3	7	7	9	12	18

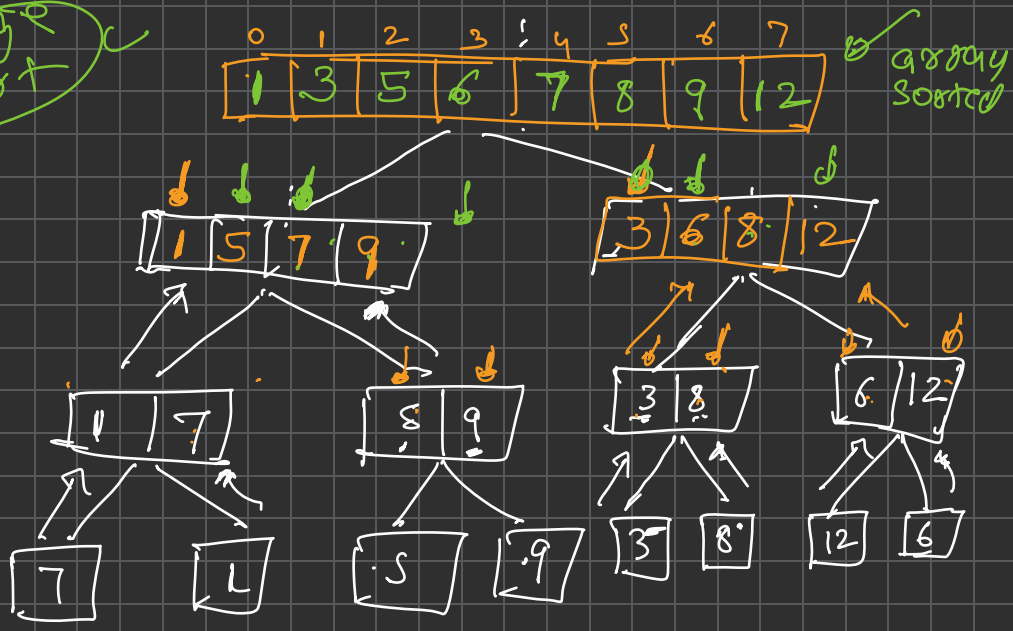


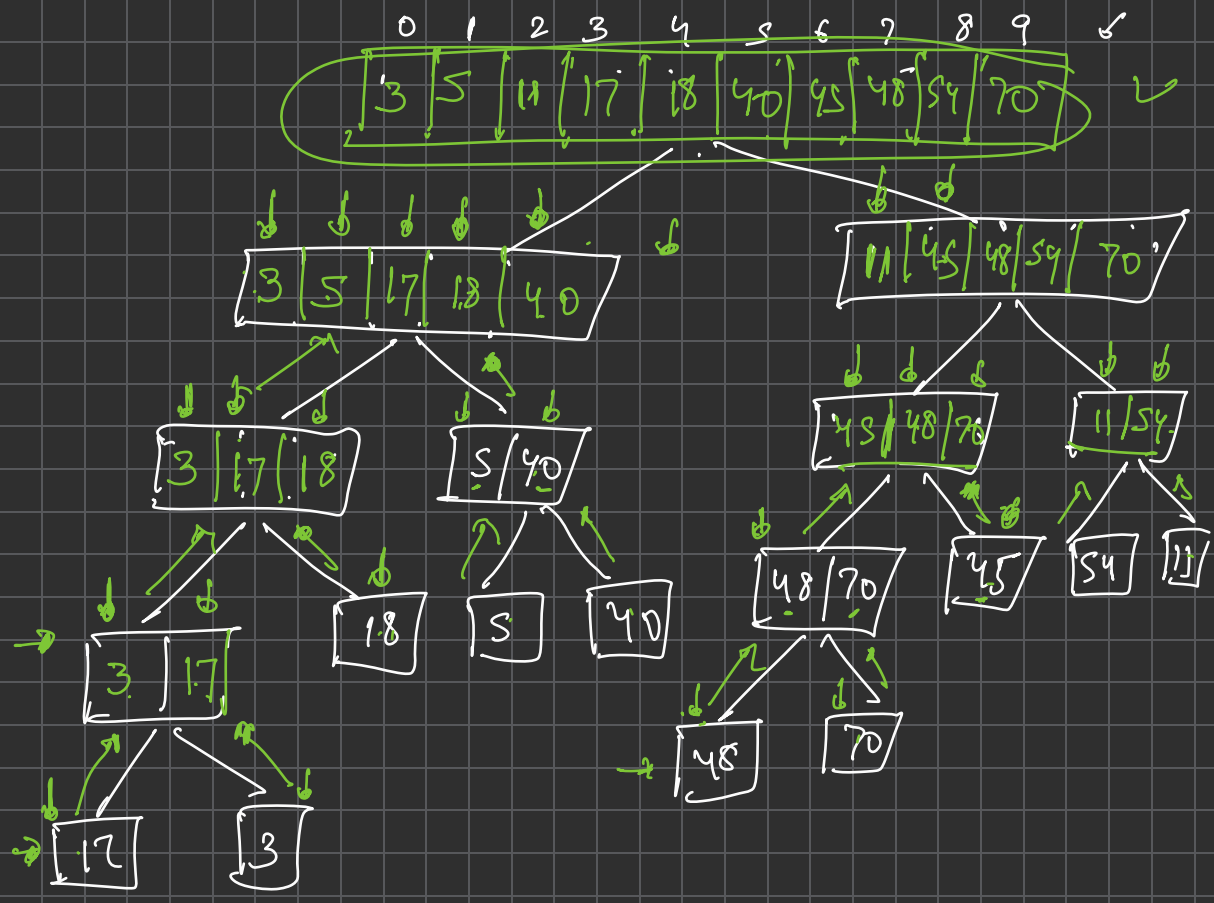

```
bool binarySearch(int arr[], int start, int end, target)
{
    if (start > end)
        return 0;

    int mid = start + (end-start)/2
    if (arr[mid] == target)
        return 1;
    else if (arr[mid] > target)
        return binarySearch(arr, start, mid-1, target)
    else
        return binarySearch(arr, mid+1, end, target)
}
```

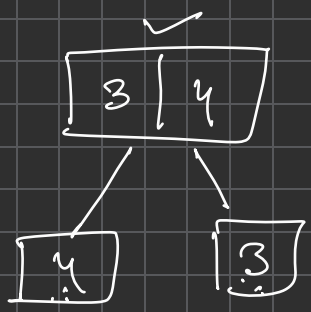

1 ⇒ true
arr, start, end, target
Right
start
arr, mid+1, end, target
Right
arr, mid+1, end, target

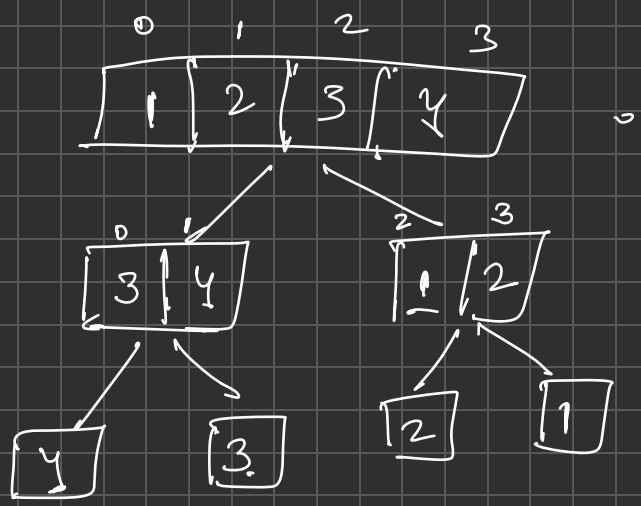
merge sort

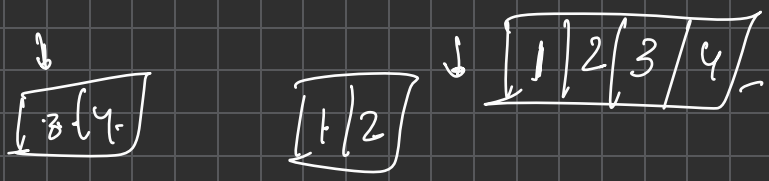

0 1 2 3 4 5 6 7
1 3 5 6 7 8 9 12
array sorted
1 5 7 9
3 6 8 12
1 7
8 9
3 8
6 12
7
1
5
9
3
8
12
6


0 1 2 3 4 5 6 7 8 9
3 5 11 17 18 40 45 48 54 70
3 5 17 18 40
11 45 48 54 70
3 17 18
5 40
45 48 70
11 54
3 17
18
5
40
48 70
45
54
11
17
3
48
70

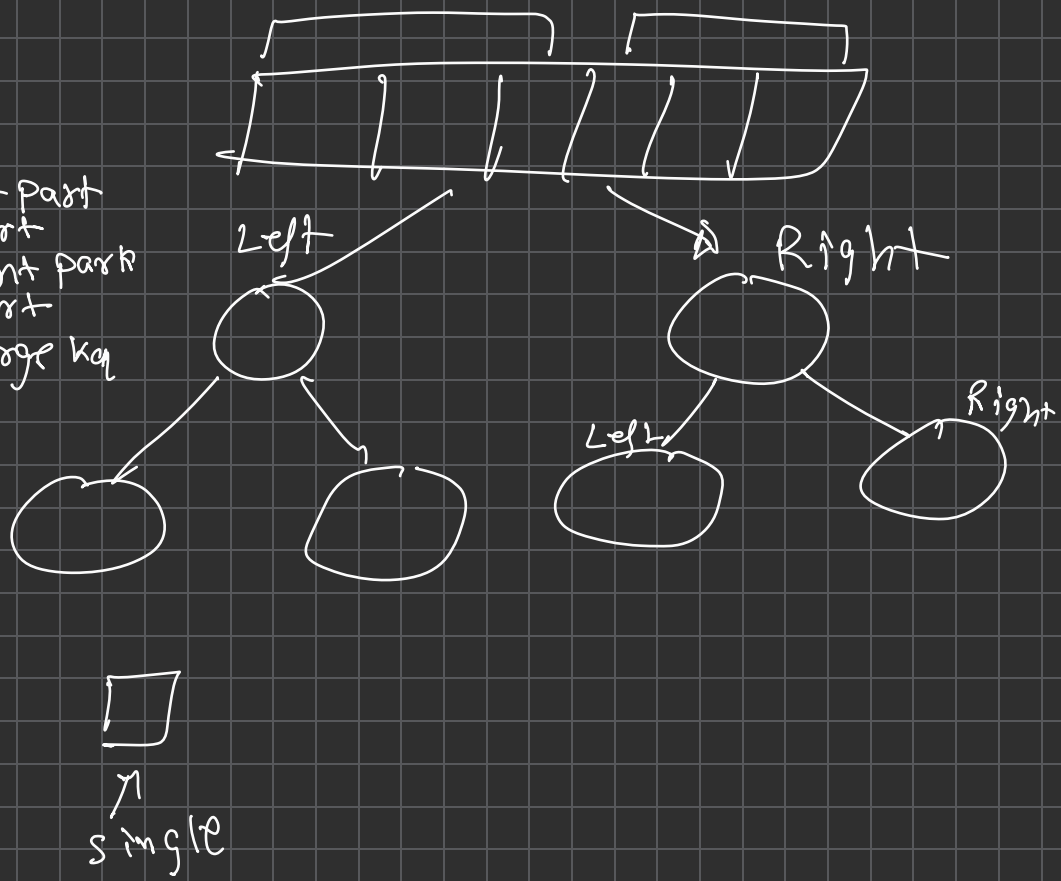
work ? ⇒ Merge-sort

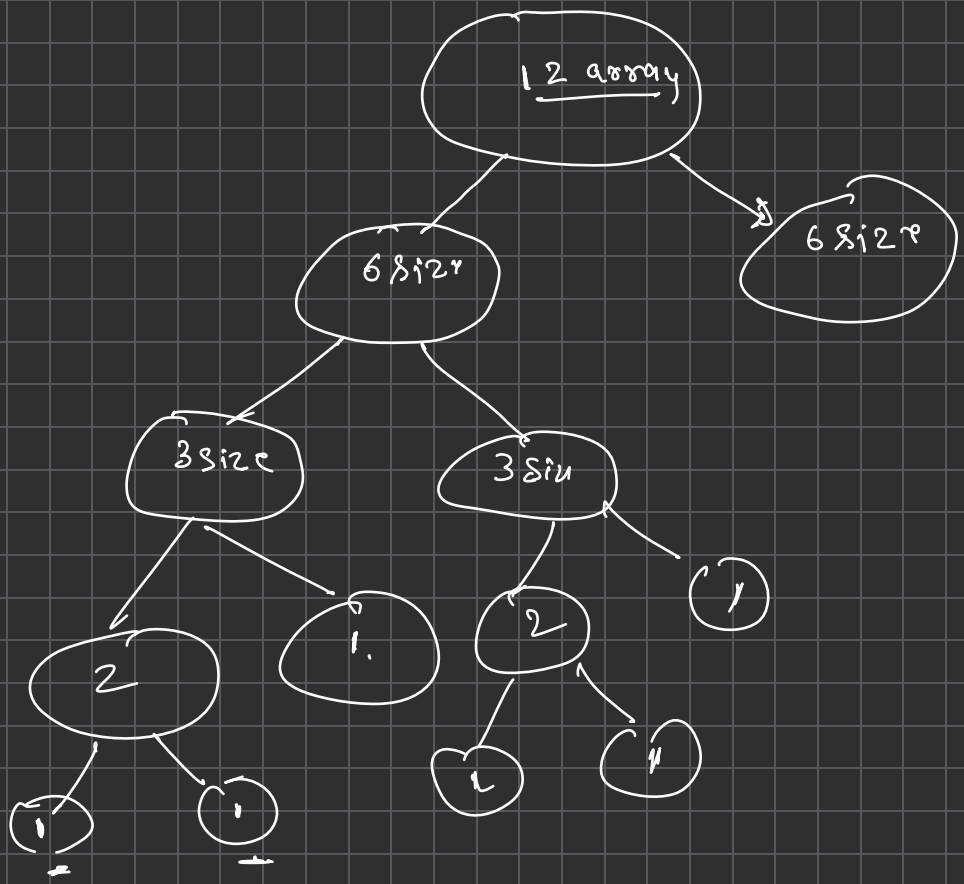






1. Left part sort
2. Right part sort
3. merge kq




12 array
6 size
6 size
3 size
3 size
2
1
2
1
1
1
1
1

array

Pivot → 8

(2) Quick-Sort:

0	1	2	3	4	5	6	7	8	9
12	18	4	17	6	3	9	5	11	8

4 | 6 | 3 | 5 | 8 | 12 | 18 | 17 | 9 | 11

4 | 3 | 5 | 6 | 9 | 11 | 12 | 18 | 17

3 | 4 | 12 | 17 | 18

sorted form

3 4 5 6 8 9 11 12 17 18

0	1	2	3	4	5	6	7	8
17	3	2	18	5	19	36	40	9

0	1	2
3	2	5

3
9

4	5	6	7	8
17	18	19	36	40

0	1
3	2

2
5

0	1
2	3

4	5	6	7
17	18	19	36

8
40

4	5	6
17	18	19

7
36

4	5
17	18

6
19

4
17

5
18